

UFOCatch Game Project

Technical Skills Portfolio

DirectX & C++ Game Development

Project Overview

Game Concept: Stealth action game where a UFO abducts animals while avoiding detection by humans.

Technology Stack:

- Language: C++
- Graphics API: DirectX 11
- Architecture: Component-based game engine
- Design Patterns: State Pattern, Object Pool, Singleton

Development Period: Personal project

Lines of Code: ~10,000+ lines of custom code

5 Key Technical Achievements

1. Spatial Partitioning Algorithm (Linear Quadtree + Morton Order)

Location: Src/09_Enemy/Liner4Tree.h

Implementation:

- Quadtree-based collision detection optimization
- Morton curve (Z-order curve) for efficient spatial indexing using bit operations
- Reduces collision check complexity from $O(n^2)$ to $O(n \log n)$

Technical Highlights:

- Implemented bit separation algorithm for Morton encoding
- Hierarchical cell traversal for spatial queries
- Template-based generic container for any game object type

```
// Morton number calculation with bit separation
uint16_t BitSeparate(uint16_t n) {
    n = (n | (n << 8)) & 0x00ff00ff;
    n = (n | (n << 4)) & 0x0f0f0f0f;
    n = (n | (n << 2)) & 0x33333333;
    n = (n | (n << 1)) & 0x55555555;
    return n;
}

uint16_t Get2DMortonNumber(float worldX, float worldY) {
    uint16_t cellX = normalizeCoord(worldX);
    uint16_t cellY = normalizeCoord(worldY);
    return BitSeparate(cellX) | (BitSeparate(cellY) << 1);
}
```

2. Geometric Collision Detection System

Location: Src/11_GameSystem/VisionSystem.h/cpp

Implementation:

- Advanced sector-circle collision detection for AI vision system
- Line segment-circle intersection using quadratic equation discriminant
- Applied to enemy AI detection mechanics

Technical Highlights:

- Mathematical approach using dot products and vector normalization
- Optimized with early rejection tests to minimize calculations
- Handles edge cases including sector boundaries and arc intersections

```
// Line segment-circle intersection detection
bool LineSegmentCircleIntersection(
    const VECTOR2& lineStart, const VECTOR2& lineEnd,
    const VECTOR2& circleCenter, float circleRadius) const {

    VECTOR2 direction = lineEnd - lineStart;
    VECTOR2 toCenter = lineStart - circleCenter;

    float a = dot(direction, direction);
    float b = 2.0f * dot(toCenter, direction);
    float c = dot(toCenter, toCenter) - radius * radius;

    float discriminant = b * b - 4.0f * a * c;
    if (discriminant < 0.0f) return false;

    float t1 = (-b - sqrt(discriminant)) / (2.0f * a);
    float t2 = (-b + sqrt(discriminant)) / (2.0f * a);

    return (t1 >= 0 && t1 <= 1) || (t2 >= 0 && t2 <= 1);
}
```

3. Design Pattern Implementation (State Pattern)

Location: Src/12_Tutorial/TutorialState.h, Src/09_Energy/Human/State/

Implementation:

- State machine for complex behavior management
- Tutorial progression: Move → Suction → Expand → Discovery → Play
- Enemy AI states: Idle → Walk → FindPlayer

Technical Highlights:

- Separation of concerns: Actor and State classes
- std::unordered_map for O(1) state lookup
- Pre-allocated states to eliminate runtime allocations
- Prevents frame rate drops during state transitions

```
// State pattern implementation
class CTutorialState {
public:
    virtual void Enter() {}
    virtual void Update() {}
    virtual void Exit() {}

    enum class State {
        Move, Suction, Expands, Discovery, Play
    };
};

class CMoveState : public CTutorialState {
    void Update() override {
        // Handle movement tutorial logic
        if (playerMoved()) transitionToNext();
    }
};

// State management with unordered_map
std::unordered_map<StateType, std::unique_ptr<State>> states;
```

4. DirectX HLSL Shader Programming

Location: Hlsl/DisplaceSkinMesh.hlsl

Implementation:

- Displacement mapping for advanced surface detail
- Skinned mesh animation supporting up to 255 bones
- Normal mapping, specular mapping for realistic rendering

Technical Highlights:

- Hull and Domain shaders for tessellation
- Vertex skinning with bone weight blending
- Tangent space calculations for normal mapping
- Optimized constant buffer usage

```
// Skinning vertex shader
struct VS_IN {
    float3 Pos : POSITION;
    float3 Normal : NORMAL;
    float2 Tex : TEXCOORD;
    uint4 Bones : BONE_INDEX;
    float4 Weights : BONE_WEIGHT;
};

Skin SkinVertex(VS_IN input) {
    Skin output = (Skin)0;

    for (int i = 0; i < 4; i++) {
        matrix boneMat = BoneFramePose[input.Bones[i]];
        float weight = input.Weights[i];

        output.Pos4 += weight * mul(float4(input.Pos, 1), boneMat);
        output.Normal += weight * mul(input.Normal, (float3x3)boneMat);
    }
    return output;
}
```

5. Game System Architecture Design

Location: Src/07_Scene/, Src/08_Player/, Src/11_GameSystem/

Implementation:

- Scene management system with lifecycle control
- Object manager for game entity lifecycle
- Game mechanics: Conical suction, experience system, Lerp interpolation

Technical Highlights:

- Frame-rate independent physics using DeltaTime
- Conical volume calculation for suction range
- Height-based suction speed using easing curves
- Internal division ratio for ground-plane projection

```
// Suction displacement calculation
VECTOR3 CalcSuctionDisplacement(float moveTime,
                                const VECTOR3& animalPos) const {
    // Calculate speed multiplier based on height
    float heightDiff = m_coneTopPos - animalPos.y;
    float progress = 1.0f - (heightDiff / m_coneTopPos);
    float eased = progress * progress * progress; // Cubic easing

    float speedMultiplier = lerp(minSpeed, maxSpeed, eased);

    // Project to ground plane using internal division
    float ratio = (0 - animalPos.y) / (animalPos.y - m_coneTopPos);
    VECTOR3 groundPoint = VECTOR3(
        animalPos.x + ratio * (animalPos.x - transform.position.x),
        0,
        animalPos.z + ratio * (animalPos.z - transform.position.z)
    );

    VECTOR3 pullVector = transform.position - groundPoint;
    return pullVector / moveTime * speedMultiplier * DeltaTime();
}
```

Summary & Impact

Code Quality:

- Used 'lizard' tool to measure cyclomatic complexity
- Maintained low complexity scores through small, focused functions
- Improved code readability, testability, and maintainability

Performance Optimization:

- Spatial partitioning reduced collision checks by ~90%
- State pre-allocation eliminated frame drops during transitions
- Frame-rate independent physics ensures consistent gameplay

Software Engineering Principles:

- Single Responsibility Principle in state design
- DRY (Don't Repeat Yourself) with template programming
- SOLID principles throughout architecture

Key Takeaways:

These implementations demonstrate proficiency in:

- Advanced C++ programming (templates, STL, modern C++11/14 features)
- Algorithm design and optimization
- Graphics programming (DirectX, HLSL)
- Software architecture and design patterns
- Performance-conscious development
- Mathematical problem-solving for game mechanics